OpenAI Debugging — Worked Example: Solution

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `openai_debugging_practice.py`: every round here is a mistake in *how* the OpenAI API is used — the prompt, the schema, the size of what you send. Each fix sits next to a test that checks it, most of them without a single API call. **If not:** you'd only find these bugs in production, one confusing 400 error at a time.

Every fix and test below goes in `practice/openai_debugging_practice.py`, and runs with `pytest openai_debugging_practice.py -v` from inside `practice/`. Only Round 1's test calls the real API (`OPENAI_API_KEY` in your `.env`); the others need no key.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: asking a chat model to "reply with JSON" in a plain prompt is a request, not a guarantee — the model is still free to add a friendly sentence around it, and often does. `json.loads()` has no tolerance for anything before the first `{`.

Story: the fix isn't a smarter `json.loads()` — it's using the feature Doc04 already taught for exactly this, so the shape is guaranteed when the answer is written, not hoped for afterwards. **If not:** you'd write string-cleaning code that breaks the first time the model phrases its preamble differently.

The fix:

```
from openai import OpenAI
from pydantic import BaseModel

client = OpenAI()

class Summary(BaseModel):
    summary: str
```

```
def summarize(user_text: str) -> Summary:
    # why: structured output makes the model write this exact shape —
    # no preamble, no parsing by hand (Doc04's Structured output topic)
    response = client.responses.parse(
        model="gpt-4o-mini",
        input=user_text,
        text_format=Summary,
    )
    return response.output_parsed
```

Test that would have caught it:

```
def test_summarize_returns_a_summary_object():
    result = summarize("Some article text about a product launch.")
    assert isinstance(result, Summary)
```

Checking the *type* of the result, not just that the call didn't raise, catches both this bug and the "valid JSON, wrong shape" case Doc02 warns about.

Round: Intermediate

Real cause: OpenAI's structured-output "strict" mode builds a JSON Schema straight from the Pydantic model, and strict mode does not allow `default` values in that schema — every field has to be required. `tags: list[str] = []` is a perfectly normal Pydantic model on its own, but its schema carries `"default": []`, so the API rejects the whole request before the model even runs.

Story: the error mentions the *schema*, not the model's answer — this round trains reading which side of the call failed. The schema can be checked in plain Python, with no API call at all. **If not:** you'd find this only by making a live call, and blame a library version.

The fix:

```
from pydantic import BaseModel

class Summary(BaseModel):
    summary: str
    # why: no default — strict mode needs every field required;
    # "no tags" is the model returning an empty list on purpose
    tags: list[str]
```

Test that would have caught it:

```
def test_summary_schema_has_no_defaults():
    schema = Summary.model_json_schema()
    for field_name in schema["properties"]:
        field_schema = schema["properties"][field_name]
        assert "default" not in field_schema, field_name
```

This test catches the mistake before any API call — checking the schema you're about to send is faster and cheaper than waiting for a `BadRequestError`.

Round: Real-world

Real cause: nothing in the chatbot ever checks the running token count before sending. Every turn appends to history and resends the whole thing. No single message is the problem — the total quietly grows every turn until one turn tips it over 128,000 tokens.

Story: a bug that needs 24 turns to appear will never show up in a 3-turn manual test. The fix is a token budget; the lesson is a test that builds a long session on purpose. **If not:** every short test would pass, and long real sessions would keep dying mid-conversation.

The fix:

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o-mini")   # Doc03's exact count

def count_tokens(message: dict) -> int:
    return len(enc.encode(message["content"]))

def build_messages_within_budget(history, system_message, max_tokens):
    running_total = count_tokens(system_message)
    kept = []
    # how: walk from the NEWEST message backwards, so the most
    # recent turns are kept and the oldest are dropped first
    for message in reversed(history):
        message_tokens = count_tokens(message)
        if running_total + message_tokens > max_tokens:
            break
        kept.append(message)
        running_total = running_total + message_tokens
    # how: kept is newest-first — flip it back into time order
    kept.reverse()
    messages = [system_message]
    messages.extend(kept)
    return messages
```

Test that would have caught it:

```
SYSTEM_MESSAGE = {"role": "system", "content": "You are a support bot."}

def build_fake_history(turn_count):
    history = []
    for i in range(turn_count):
        text = "This is turn number " + str(i) + " of a long chat. " * 20
        history.append({"role": "user", "content": text})
    return history

def test_long_session_stays_within_token_budget():
    history = build_fake_history(turn_count=200)
    messages = build_messages_within_budget(
        history, SYSTEM_MESSAGE, max_tokens=2000
    )
    total = 0
    for message in messages:
        total = total + count_tokens(message)
    assert total <= 2000
    # the newest turn must survive the trimming
    assert messages[-1] == history[-1]
```

A small `max_tokens` (2,000 instead of 120,000) lets 200 fake turns overflow it quickly — same logic, a test that runs in a moment.

Round: Multi-agent

Real cause: every agent writing its full scratchpad into shared state means each later agent's prompt is built from everyone-before-it's raw reasoning, not their finished conclusions. This is the shared-state design mistake Doc11's Core Concepts names — too much in shared state buries what a later agent needs under noise, and sometimes pushes the request over the same token limit as Round 3, this time from 4 agents' scratchpads instead of one long chat.

Story: testing the Reviewer with a small, clean, hand-made state hides this completely. The fix is deciding what each node is allowed to write; the test checks exactly that, on the node's real output. **If not:** shared state would keep growing with every agent, and quality would quietly drop on longer tasks.

The fix:

```
from typing import TypedDict
from langgraph.types import Command
```

```
class SharedState(TypedDict):
    task: str
    research_findings: str   # Research agent's finished output only
    analysis: str            # Analysis agent's finished output only
    draft: str               # Writer agent's finished output only

def run_research_tools(task):
    # stand-in: a long raw scratchpad, like real tool calls produce
    return "step: searched the web, got 10 results...\n" * 500

def summarize_findings(raw_scratchpad):
    # stand-in for the Research agent's own summary call
    return "Refunds are allowed within 30 days of purchase."

def research_node(state):
    raw_scratchpad = run_research_tools(state["task"])
    finished_summary = summarize_findings(raw_scratchpad)
    # why: raw_scratchpad stays local to this node — only the
    # finished summary goes into shared state
    return Command(
        goto="analysis_agent",
        update={"research_findings": finished_summary},
    )
```

Test that would have caught it:

```
def test_research_node_writes_only_its_finished_output():
    result = research_node({"task": "refund policy"})
    written_keys = list(result.update.keys())
    assert written_keys == ["research_findings"]
    assert len(result.update["research_findings"]) < 1000
```

The test runs the real node and checks exactly what it writes into shared state — its keys and its size — not only whether the pipeline finished without an error.

Real cause vs. symptom fix

A symptom fix would be catching `JSONDecodeError` and retrying with a slightly different prompt, hoping for cleaner output (Round 1), removing the `tags` field entirely instead of understanding strict mode's rule (Round 2), catching the context-length error and blindly dropping the oldest half of history with no budget logic (Round 3), or cutting every shared-state field at a fixed number of characters instead of deciding what belongs there at all (Round 4). Each of those makes today's failure quiet without addressing why it happens — which is Core Concepts' point about checking a fix

by understanding it, not by the symptom going away. A blunt cut, in particular, removes content at an arbitrary point instead of the *right* point, trading one confusing failure for a quieter one.